LABORATORIUM ALGORITMA & PEMROGRAMAN

PROGRAM STUDI TEKNIK INFORMATIKA

Mata Kuliah: Praktikum Algoritma & Pemrograman (KK112601) • Semester Ganjil 2024/2025

LEMBAR KERJA PRAKTIKUM (LKP) MAHASISWA

Modul Praktikum	: **Modul 01 – Tipe Data, Variabel & Operator**	**Tanggal Uji**	: 15 Oktober 2026
Nama Mahasiswa	: **SITI AMINAH**	**Kelas**	: **TI-SP1**
NIM	: `1126100102`	**Dosen Pengampu**	: **Hadiq, ST, M.Kom**

A. CAPAIAN PRAKTIKUM (LEARNING OUTCOMES)

1. Mampu mengidentifikasi karakteristik domain nilai dan alokasi memori fisik per tipe data (integer, float, string, boolean) pada memori komputer.
2. Mampu membuktikan secara empiris fenomena pemotongan desimal (integer division truncation), konversi tipe data (type casting), dan perilaku string concatenation bug.
3. Mampu menganalisis implikasi kesalahan pemilihan tipe data terhadap kebenaran logika dan akurasi presisi komputasi program.

B. DASAR TEORI & HIPOTESIS UJI LABORATORIUM

Variabel adalah label pengenal yang merujuk pada alamat lokasi fisik tertentu di dalam Random Access Memory (RAM). Ketika variabel dideklarasikan, komputer mengalokasikan sejumlah byte memori fisik berdasarkan Tipe Data yang ditentukan.

#1 KAPASITAS DOMAIN NILAI & AKURASI PRESISI KOMPUTASI `Domain & Presisi RAM`

Mengendalikan batas rentang nilai (domain) minimum dan maksimum yang dapat ditampung, serta ketepatan presisi fraksi desimal di memori (seperti mantissa 52–bit pada standar IEEE 754 vs integer murni).

#2 KEABSAHAN OPERASI MATEMATIKA & LOGIKA `Type Safety & Operasi`

Mengendalikan validitas instruksi yang diizinkan dieksekusi prosesor (misalnya pembedaan operator + untuk penjumlahan numerik vs penggabungan teks str, serta proteksi terhadap runtime error fatal seperti ZeroDivisionError).

1. ALOKASI MEMORI RAM & KARAKTERISTIK TIPE PRIMITIF

Skenario Uji #1 & #2

Intisari: **Variabel bukan sekadar penampung nilai statis, melainkan pointer (referensi alamat) ke memori fisik RAM dengan pembungkus struktur objek PyObject.**

- **Hakikat Variabel:** Variabel bekerja sebagai pointer yang menunjuk ke alamat memori fisik RAM tempat data disimpan.
- **Struktur Objek (PyObject):** Tipe primitif dibungkus sebagai objek lengkap dengan header referensi (refcount), penanda tipe data, dan nilai. `[PyObject]`
- **Alokasi Memori Fisik:** Bilangan bulat int memerlukan alokasi 28 byte di RAM, sedangkan pecahan float memerlukan 24 byte berstandar IEEE 754. `[int: 28B | float: 24B]`
- **Fungsi Diagnostik Memori:** Gunakan type() untuk memvalidasi kelas tipe data dan sys.getsizeof() untuk mengukur kapasitas byte fisik variabel di RAM. `[type() & sys.getsizeof()]`

2. PEMBAGIAN PECAHAN (/) VS PEMOTONGAN DESIMAL (//) (INTEGER TRUNCATION)

Skenario Uji #3 & #4

Intisari: **Komputer membedakan secara tegas antara pembagian pecahan riil (/) yang mempertahankan desimal, dan pembagian bulat (//) yang memicu pemotongan desimal paksa (integer truncation).**

- **Float Division (/):** Operasi 10 / 4 menghasilkan nilai 2.5 bertipe float dengan seluruh presisi angka di belakang koma dipertahankan utuh. `[10 / 4 -> 2.5]`

- **Floor Division (//):** Operasi 10 // 4 menghasilkan nilai 2 bertipe int; seluruh angka di belakang koma dibuang paksa tanpa pembulatan. `[10 // 4 -> 2]`
- **Integer Truncation:** Bagian pecahan 0.5 dimusnahkan seketika dari memori kalkulasi, bukan dibulatkan ke atas (bukan ceiling).
- **Bahaya Akurasi Finansial:** Ketidaksengajaan memakai operator // pada sistem perbankan atau formula sains memicu akumulasi selisih yang fatal.

3. KARAKTERISTIK MASUKAN KEYBOARD & KEWAJIBAN EXPLICIT TYPE CASTING

Skenario Uji #5 & #6

Intisari: **Seluruh data masukan dari input() selalu bertipe String (str), sehingga operator + memicu penggabungan teks (concatenation) bukan operasi aritmatika tanpa Explicit Type Casting.**

- **Sifat Bawaan input():** Semua karakter yang diketikkan pengguna ditangkap sistem operasi sebagai deretan teks bertipe String (str). `[type -> str]`
- **String Concatenation Bug:** Operasi '50' + '25' menghasilkan teks gabungan '5025' (bukan 75). Program berjalan lancar tanpa pesan error (silent bug). `['50' + '25' -> '5025']`
- **Explicit Type Casting:** Wajib membungkus variabel dengan fungsi int() atau float() sebelum variabel dioperasikan secara matematis. `[int('50') + int('25') -> 75]`
- **Pencegahan Silent Error:** Validasi konversi tipe sedini mungkin mencegah kebocoran kesalahan logika ke tahapan kalkulasi berikutnya.

4. PENANGANAN INTERUPSI FATAL: PEMBAGIAN DENGAN NOL (DIVISION BY ZERO)

Skenario Uji #7

Intisari: **Pembagian dengan pembagi nol (x / 0) adalah operasi tak terdefinisi yang membangkitkan interupsi hardware trap dan menghentikan eksekusi dengan ZeroDivisionError.**

- **Operasi Tak Terdefinisi:** Secara matematis pembagian dengan nol tidak terdefinisi dan tidak dapat diproses oleh unit logika aritmatika (ALU).
- **Hardware Trap & Crash:** Mikroprosesor membangkitkan interupsi perangkat keras yang memaksa sistem operasi menghentikan proses runtime secara mendadak. `[ZeroDivisionError]`
- **Defensive Precondition:** Wajib menyertakan percabangan validasi if pembagi != 0 sebelum instruksi pembagian diizinkan dieksekusi. `[if pembagi != 0:]`
- **Ketahanan Sistem Produksi:** Membuktikan pentingnya sanitasi masukan agar aplikasi tangguh dan tidak rentan crash akibat masukan tidak valid.

5. KETERBATASAN PRESISI BINER PECAHAN (STANDAR IEEE 754 & ROUNDING ARTIFACT)

Skenario Uji #8

Intisari: **Pecahan desimal seperti 0.1 dan 0.2 membentuk biner berulang tak hingga pada format IEEE 754 64-bit, memicu rounding artifact (0.1 + 0.2 != 0.3) akibat batas mantissa 52-bit.**

- **Format IEEE 754 64-Bit:** Bit memori dibagi menjadi 1 bit sign, 11 bit exponent, dan 52 bit mantissa fraksi. `[Mantissa 52-bit]`
- **Rounding Artifact Terkenal:** Operasi 0.1 + 0.2 di memori menghasilkan 0.30000000000000004 akibat pemotongan biner berulang tak hingga. `[0.30000000000000004]`
- **Kegagalan Uji Kesamaan ==:** Pengujian 0.1 + 0.2 == 0.3 mengembalikan False karena terdapat selisih pecahan residual pada digit ke-17. `[== 0.3 -> False]`
- **Solusi Rekayasa Epsilon:** Gunakan toleransi selisih epsilon abs(a - b) < 1e-9, pembulatan round(), atau fungsi math.isclose(a, b). `[math.isclose(a, b)]`

6. DOMAIN TIPE BOOLEAN, OPERATOR RELASIONAL & VALIDASI LOGIKA

Status Validasi Tabel C

Intisari: **Tipe Boolean (bool) merepresentasikan status diskrit (True / False) hasil evaluasi Operator Relasional sebagai fondasi percabangan dan instrumen validasi pengujian.**

- **Domain Diskrit Biner:** Hanya memiliki dua status kebenaran mutlak: True (benar / 1 logika) atau False (salah / 0 logika). `[True | False]`
- **Operator Relasional:** Dievaluasi melalui perbandingan kesamaan (==), ketidaksamaan (!=), dan rentang (<, <=, >, >=). `[==, !=, <, >]`
- **Perangkap = vs ==:** Tanda = adalah operator penugasan nilai (assignment), sedangkan == adalah operator pengujian kesamaan nilai (equality).
- **Instrumen Validasi Lab:** Digunakan pada kolom status validasi untuk membuktikan kesesuaian antara luaran aktual mesin dengan hipotesis teoritis.

C. TABEL PENGAMATAN & PENGUKURAN EMPIRIS

Instrumen Pencatatan Hasil Uji Komputasi

No	Skenario Pengujian	Masukan / Kode Uji	Prediksi Teori	Hasil Aktual Komputer	Metrik Terukur	Validasi
1	Pengukuran Alokasi & Tipe Data Integer	`a = 100; type(a)`	Tipe int, bilangan bulat diskrit	**<class 'int'>, 100**	28 byte (Python int object)	**Sesuai**
2	Pengukuran Alokasi & Tipe Data Float	`b = 100.5; type(b)`	Tipe float, bilangan pecahan presisi	**<class 'float'>, 100.5**	24 byte (IEEE 754 double)	**Sesuai**

No	Skenario Pengujian	Masukan / Kode Uji	Prediksi Teori	Hasil Aktual Komputer	Metrik Terukur	Validasi
3	Uji Pembagian Pecahan (Float Division)	`hasil = 10 / 4`	Menghasilkan 2.5 bertipe float	**2.5 (tipe float)**	Presisi desimal dipertahankan	**Sesuai**
4	Uji Pemotongan Desimal (Integer Truncation)	`hasil = 10 // 4`	Pecahan dibuang, menghasilkan 2	**2 (tipe int)**	0.5 desimal terpotong (hilang)	**Sesuai**
5	Masukan String Tanpa Type Casting	`x = '50'; y = '25'; z = x + y`	Terjadi penggabungan teks '5025'	**'5025' (tipe str)**	Operasi string concatenation	**Sesuai**
6	Masukan dengan Explicit Type Casting	`x = int('50'); y = int('25'); z = x + y`	Penjumlahan aritmatika bernilai 75	**75 (tipe int)**	Operasi aritmetika numerik	**Sesuai**
7	Uji Kasus Ekstrem: Pembagian dengan Nol	`hasil = 100 / 0`	Sistem menolak / memicu exception	**ZeroDivisionError: division by zero**	Program berhenti (Crash / Interrupted)	**Sesuai**
8	Uji Presisi Floating Point (IEEE 754)	`0.1 + 0.2`	Bernilai 0.3	**0.30000000000000004**	Penyimpangan biner IEEE 754	**Sesuai**

D. ANALISIS & PEMBAHASAN HASIL PENGUKURAN

1. Jelaskan mengapa hasil dari operasi 10 // 4 membuang nilai desimal, dan apa implikasi fatalnya jika operator ini secara tidak sengaja digunakan dalam sistem perhitungan diskon atau perbankan!

> *hasil operasi 10 // 4 tidak mempunyai nilai desimal dibelakang koma karena operator // adalah operator pembagi bulat sama dengan DIV yang memberikan hasil bilangan bertipe integer atau bulat. Jika tidak sengaja digunakan dalam sistem perhitungan diskon atau perbangkan yang seharusnya menggunakan float, maka angka desimal dibelakang koma akan hilang dan bisa mengajibatkan kerugian untuk jumlah perhitungan yang banyak.*

2. Berdasarkan hasil pengujian pada skenario 5 dan 6, jelaskan mengapa input dari pengguna secara default dianggap sebagai teks (string), serta mengapa type casting menjadi kewajiban mutlak sebelum melakukan kalkulasi!

> *Input dari pengguna secara default adalah string karena keyboard sebagai piranti input tidak hanya mempunyai tombol angka tapi juga mempunyai huruf dan karakter khusus. sehingga tipe data yang paling alami sesuai terhadap masukan dari tombol keyboard adalah string atau alfanumerik. jika kita perlu melakukan perhitungan secara numerik maka kita perlu melakukan type casting dari string ke integer atau float agar hasil perhitungan mempunyai hasil hal sesuai. contoh 1 + 1 = 2 bukan '1' + '1' = '11'*

3. Jelaskan temuan Anda pada skenario 8 mengenai hasil 0.1 + 0.2 = 0.30000000000000004! Mengapa fenomena tersebut terjadi di dalam memori komputer dan bagaimana seorang programmer mengantisipasinya?

penjumlahan 0.1 + 0.2 = 0.300000000000004 dikarenakan perbedaan representasi angka antara komputer dan manusia. manusia menmggunakan sistem bilangan desimal yang berbasis sepulu, sehingga dengan mudah memberikan representasi 0.1 + 0.2 = 0.3. Sedangkan komputer yang menggunakan sistem bilangan biner mempunyai tingkat kerumitan yang tinggi pada saat merepresentasikan angka pecahan. hal ini mempengaruhi tingkat presisi dalam menulis desimal dibelakang koma. dengan kondisi ini maka programmer harus mengantisipasi dengan meberikan format berapa desimal dibelakang koma yang perlu ditampilkan

E. KESIMPULAN ILMIAH

1. Ketepatan dalam menentiuka tipe data sangat penting dalam pemrograma, karena jika salah dalam menentukan tipe data maka akan berpotensi memverikan kesalahan perhitungan. 2. setiap kali akan melakukan operasi aritmatika pada data yang berasal dari inputan maka harus dilakukan konversi dari tipe data string ke int atau float agar hasil perhitungan memberikan nlai yang valid. 3. Karena representasi angka antara komputer dan manusia berbeda, maka terjadi akurasi yang kurang tepat ketika komputer melakukan operasi bilangan pecahan atau float. untuk itu maka programmer harus memberikan format berapa desimal yang perlu diambil dari hasil perhirungan bilangan pecahan agar mempunyai konsistensi dalm hasil perhitungan

AI PRE-GRADING VERIFIED	Skor Kelayakan Ilmiah: **85 / 100** (Memenuhi Standar)	Audit Sistem: GROQ (QWEN3.8) • 19.10

LEMBAR PENILAIAN & PENGESAHAN LABORATORIUM
Mata Kuliah Praktikum Algoritma & Pemrograman (1 SKS)

Nilai / Skor Akhir : **85 / 100**

Menyatakan Kebenaran Hasil Uji,
Mahasiswa Praktikan,



SITI AMINAH
NIM. 1126100102

Menyetujui & Mengesahkan,
Dosen Pengampu Laboratorium,



Tanda Tangan Digital Sah • 15/10/2026

Hadiq, ST, M.Kom
NIDN: 0713077804